

C2. We Be Flipping (Hard Version)

题意

给定长度为 n 的数组 $a_1, a_2, \dots, a_n$ 。每次操作可以选择一个位置 i ，将前缀 $[1..i]$ 的所有元素取反（乘 -1 ）。要求用**最少**的操作次数，使得最终数组中所有元素**同号**（全 ≤ 0 或全 ≥ 0 ）。

若初始数组已同号，答案为 0。

解题思路

Easy Version (C1) — 任意可行解

从右向左贪心：维护 $flip$ 表示当前位置被后续操作翻转的奇偶性。对于位置 i ：

- 当前有效值 $cur = flip ? -a_i : a_i$
- 若 $cur > 0$ ，则必须在 i 处翻转一次（使该位置 ≤ 0 ）， $flip \wedge= 1$

该贪心一定给出一个合法解，将所有元素变为 ≤ 0 。

Hard Version (C2) — 最小操作次数

核心在于：**并非所有正数都需要单独翻转**。通过选择合适的翻转位置，一次翻转可以同时修正多个元素。

算法步骤：

1. 前缀负权值和 $f[i]$ ：

$$f[i] = \sum_{j=1}^i |a_j| \cdot [a_j < 0]$$

即前缀中所有负数的绝对值之和。

2. 寻找最优分割点 $index$ ：

遍历 $i \in [1, n-1]$ ，若 $a_{i+1} > 0$ ，计算 $f[i] - a_{i+1}$ 。取使该值最大的 $i+1$ 作为 $index$ 。

直观理解： $f[i]$ 代表「前面的负数质量」， a_{i+1} 是当前位置的正数值。 $f[i] - a_{i+1}$ 越大，说明在该处翻转的「净收益」越高。

3. **判定无解**：若 $index == 0$ （不存在使 $f[i] - a_{i+1} > 0$ 的位置），说明数组已经满足条件，输出 0。

4. **构造方案**：

- 对位置 $[index - 1, 1]$ 从右向左贪心翻转（同 C1）
- 最后在 $index$ 处追加一次翻转
- 位置 $> index$ 的元素无需处理（已合法）

复杂度

- 时间复杂度： $O(n)$
- 空间复杂度： $O(n)$

关键代码片段

```
// 前缀负权值和
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    if (a[i] < 0)
        f[i] = abs(a[i]) + f[i - 1];
    else
        f[i] = f[i - 1];
}

// 寻找最优分割点
int index = 0, mx = 0;
for (int i = 1; i < n; i++) {
    if (a[i + 1] > 0) {
        if (f[i] - a[i + 1] > mx) {
            index = i + 1;
            mx = f[i] - a[i + 1];
        }
    }
}

// 从右向左构造答案
int cnt = 0, flip = 0;
for (int i = index - 1; i >= 1; i--) {
    int cur = flip ? -a[i] : a[i];
    if (cur > 0) {
        ans[++cnt] = i;
        flip ^= 1;
    }
}
ans[++cnt] = index;
```

注意事项

- $f[i]$ 可能很大，需使用 long long（代码中使用 long long f[N]）
- mx 初值为 0，只有严格 > 0 才更新 index，保证不会无意义翻转
- C1 与 C2 的核心区别：C2 通过前缀分析跳过了尾部已合法的元素，减少了不必要的翻转